Project Work Summary

Date: 2026-01-17

Prepared by: GitHub Copilot (session actions recorded)

---

1) Purpose

This document summarizes the repository `tomato-diseace-identification` and the work performed during this session to collect and prepare a project-level report.

2) Repository Overview

- Purpose: Flask web application to classify tomato leaf diseases from uploaded images. Also includes an Expo React Native mobile client and an Android native app wrapper.
- Primary frameworks/libraries: `Flask`, `TensorFlow`, `Pillow`, `NumPy`.
- High-level features:
  - Web UI for image upload and classification (`/` endpoint)
  - JSON API (`/api/predict`) for mobile apps
  - Health endpoint (`/health`)
  - Optional leaf-detector model to reject non-leaf images
  - Admin UI and API to tune HSV thresholds
  - Mask overlay generation showing green portions of images
  - Docker, Heroku deployment instructions and CI workflows

3) Key files inspected

- `README.md` — project README and setup instructions.
- `requirements.txt` — dependency list (note: `tensorflow` not pinned).
- `app.py` — main Flask application; contains API endpoints, preprocessing, heuristic leaf detection, optional leaf detector, and mask overlay generation.
- `tests/test_api_health.py` — simple health check test that expects a running server.
- `mobile_app/App.js` — Expo React Native app connecting to `/api/predict`.
- Android assets: `mobile_app_android/app/src/main/assets/` contains `labels.txt` and a tflite model in some branches.
- CI and workflow configs under `.github/workflows`.

File locations and structure were enumerated from the workspace.

4) Notable code & repository observations

- Merge conflict markers found in multiple files (e.g., `README.md`, `requirements.txt`, `app.py`) indicating unresolved git conflicts — these should be resolved before production.
- `requirements.txt` includes `tensorflow` without a pinned version; this can cause reproducibility issues and long installs. Pinning TensorFlow (and CPU/GPU variant) is recommended.
- `app.py` contains both heuristic HSV-based leaf checks and an optional `leaf_detector` model; the code lazy-loads models at first request.
- Debugging helpers and debug-output logic exist (writes JSONL debug log and copies uploads to `static/uploads/debug/`). Consider toggling or securing debug mode for production.
- The web upload and API endpoints both perform leaf checks and model predictions, but there are slight behavioral differences across merged versions (see conflict blocks).

5) How to run (development)

1. Create and activate a virtual environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

2. Install dependencies:

```
.\.venv\Scripts\pip.exe install -r requirements.txt
```

3. Place or obtain a trained Keras model `models/tomato_model.h5` (expected input 224x224x3, 10 classes).

4. Run the app:

.\.venv\Scripts\python.exe app.py

5. Open the UI at `http://localhost:5000`. Health check available at `http://localhost:5000/health`.

6) Tests

- The included test `tests/test_api_health.py` uses `requests` to GET `/health`. The server must be running for the test to pass.

7) Mobile app notes

- `mobile_app/App.js` must be updated: set `BACKEND_URL` to your server address and port.
- The app uploads images to `/api/predict` and expects JSON `{ filename, prediction, confidence, mask }`.
- `mobile_app_android` contains a native wrapper and assets (possible `inception_v3_299.tflite` in legacy paths).

8) Actions taken in this session

1. Created a short plan (tracked via the workspace todo list).
2. Scanned repository tree and enumerated files.
3. Read and extracted contents of key files for the report (`README.md`, `requirements.txt`, `app.py`, `tests/test_api_health.py`, `mobile_app/App.js`).
4. Noted unresolved merge conflict markers and other issues.
5. Drafted this Markdown summary and generated a PDF (see output file).

9) Recommendations / Next steps

- Resolve merge conflicts in `app.py`, `README.md`, and `requirements.txt`.
- Pin `tensorflow` and other major dependencies in `requirements.txt` for reproducible installs.
- Add a small CI job to run `pytest` after the server is started in a test container, or mock the `/health` call.
- Consider packaging the model download step or adding instructions for placing the model in `models/`.
- Review debug logging (JSONL) and ensure sensitive data is not written to repo or exposed.

Generated artifacts:
- `project_work_summary.md` (this file)
- `project_work_summary.pdf` (generated by the converter script)

If you want additional details (file-by-file diffs, a larger PDF with full file contents appended, or commit-level history), tell me and I will include those.

Appendix: Full contents of key files

Below are the full contents (as read during this session) of the most relevant files: `README.md`, `requirements.txt`, `app.py`, `tests/test_api_health.py`, and `mobile_app/App.js`.

`README.md`

■ Tomato Disease Classification Web App

A Flask-based web application that uses deep learning to classify tomato leaf diseases from uploaded images.

Features

- Upload and classify tomato leaf images
- Predicts 10 different tomato plant conditions
- Real-time image processing and classification

- Clean, responsive web interface
- Displays prediction confidence scores

Project Structure

```
tomato_disease_app/
■
■■■ app.py                # Flask backend code
■■■ requirements.txt      # List of dependencies
■■■ models/               # Folder to store trained model
■   ■■■ README.md         # Model placement instructions
■■■ dataset/              # Empty dataset folder
■   ■■■ train/
■   ■■■ val/
■■■ templates/
■   ■■■ index.html        # HTML template
■■■ static/
■   ■■■ style.css         # CSS styling
■   ■■■ uploads/          # Uploaded images stored here
```

Setup Instructions

1. Create a virtual environment (recommended):
   ```
   python -m venv venv
   .\venv\Scripts\activate  # Windows
   source venv/bin/activate # Linux/Mac
   ```

2. Install dependencies:
   ```
   pip install -r requirements.txt
   ```

3. Place your trained model:
   - Put your trained Keras model file `tomato_model.h5` in the `models/` directory
   - The model should be configured for input shape (224, 224, 3)
   - Should predict 10 classes: Bacterial spot, Early blight, Late blight, etc.

4. Run the application:
   ```
   python app.py
   ```

5. Access the web interface:
   - Open your browser and navigate to `http://localhost:5000`
   - The health check endpoint is available at `http://localhost:5000/health`

Usage

1. Open the web interface in your browser
2. Click "Choose File" to select a tomato leaf image
3. Click "Analyze Image" to get the prediction
4. View the results showing:
   - Uploaded image
   - Predicted condition
   - Confidence score

<<<<<<< HEAD
Mobile app (Expo)

An Expo React Native app is included in `mobile_app/`.

Quick start:

```
cd mobile_app
npm install
npm start
```

Edit `mobile_app/App.js` and set `BACKEND_URL` to point to your running Flask server, e.g. `http://192.168.1.10:5000`.

On-device inference (optional)

You can convert the provided Keras model to TensorFlow Lite for on-device inference using:

python convert_to_tflite.py

This writes `models/tomato_model.tflite` which can be integrated into native Android/iOS
or used with TensorFlow Lite interpreters.

Running tests

Run the simple health check test (make sure the server is running):

pip install pytest
pytest tests/test_api_health.py

=======
>>>>>>> a17aef9ddcd5f1388b27f3a97ec1ef9a16beaa98

`requirements.txt`

flask==2.0.1
tensorflow
numpy
pillow
werkzeug==2.0.2
<<<<<<< HEAD
requests
flask-cors==3.0.10
=======
requests
>>>>>>> a17aef9ddcd5f1388b27f3a97ec1ef9a16beaa98

`app.py` (excerpted full contents as read)

import os
from flask import Flask, request, render_template, jsonify
<<<<<<< HEAD
from flask_cors import CORS
=======
>>>>>>> a17aef9ddcd5f1388b27f3a97ec1ef9a16beaa98
from werkzeug.utils import secure_filename
import numpy as np
from PIL import Image
import tensorflow as tf
<<<<<<< HEAD
import logging
import json
import shutil

app = Flask(__name__)
CORS(app)
=======

app = Flask(__name__)
>>>>>>> a17aef9ddcd5f1388b27f3a97ec1ef9a16beaa98

# Configure upload folder and allowed extensions
UPLOAD_FOLDER = 'static/uploads'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg'}
MODEL_PATH = 'models/tomato_model.h5'
# Optional leaf-detector model (binary: leaf / not-leaf)
LEAF_DETECTOR_PATH = 'models/leaf_detector.h5'

Configurable HSV thresholds and green proportion via env (defaults tuned)

```
GREEN_H_MIN = int(os.getenv('GREEN_H_MIN', 25))
GREEN_H_MAX = int(os.getenv('GREEN_H_MAX', 100))
S_MIN = int(os.getenv('S_MIN', 40))
V_MIN = int(os.getenv('V_MIN', 40))
GREEN_PROP_THRESH = float(os.getenv('GREEN_PROP_THRESH', 0.03))

<<<<<<< HEAD
# Prediction confidence threshold (below this -> mark as 'Uncertain')
CONF_THRESH = float(os.getenv('CONF_THRESH', 0.6))

=======
>>>>>>> a17aef9ddcd5f1388b27f3a97ec1ef9a16beaa98
# Persisted config file for admin-tuned thresholds
CONFIG_PATH = 'config.json'

def load_config():
    if os.path.exists(CONFIG_PATH):
        try:
            import json
            with open(CONFIG_PATH, 'r') as f:
                cfg = json.load(f)
            return cfg
        except Exception:
            return {}
    return {}

def save_config(cfg):
    try:
        import json
        with open(CONFIG_PATH, 'w') as f:
            json.dump(cfg, f)
    except Exception as e:
        print(f"Error saving config: {e}")

# Load persisted overrides (if any)
_cfg = load_config()
GREEN_H_MIN = int(_cfg.get('GREEN_H_MIN', GREEN_H_MIN))
GREEN_H_MAX = int(_cfg.get('GREEN_H_MAX', GREEN_H_MAX))
S_MIN = int(_cfg.get('S_MIN', S_MIN))
V_MIN = int(_cfg.get('V_MIN', V_MIN))
GREEN_PROP_THRESH = float(_cfg.get('GREEN_PROP_THRESH', GREEN_PROP_THRESH))

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024  # 16MB max file size

<<<<<<< HEAD
# Debug helpers
DEBUG_DIR = os.path.join(UPLOAD_FOLDER, 'debug')
DEBUG_LOG = os.path.join(DEBUG_DIR, 'debug_logs.jsonl')
os.makedirs(DEBUG_DIR, exist_ok=True)

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')
logger = logging.getLogger(__name__)

=======
>>>>>>> a17aef9ddcd5f1388b27f3a97ec1ef9a16beaa98
# Disease class labels
class_labels = [
    'Bacterial_spot', 'Early_blight', 'Late_blight',
    'Leaf_Mold', 'Septoria_leaf_spot', 'Spider_mites',
    'Target_Spot', 'Yellow_Leaf_Curl_Virus',
    'Mosaic_virus', 'Healthy'
]

---
```

`tests/test_api_health.py`

```
import requests


def test_health():
    resp = requests.get('http://localhost:5000/health')
    assert resp.status_code == 200
    data = resp.json()
    assert data.get('status') == 'ok'
```

`mobile_app/App.js`

```
import React, { useState } from 'react';
import { StyleSheet, View, Button, Image, Text, ActivityIndicator, Alert, ScrollView }
from 'react-native';
import * as ImagePicker from 'expo-image-picker';

// IMPORTANT: update this to your backend address (include port), e.g.
http://192.168.1.10:5000
const BACKEND_URL = 'http://YOUR_BACKEND_HOST:5000';

export default function App() {
  const [image, setImage] = useState(null);
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState(null);
  const [maskUrl, setMaskUrl] = useState(null);

  async function pickImage() {
    const permission = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (!permission.granted) {
      Alert.alert('Permission required', 'Camera roll permission is required to pick
images.');
      return;
    }

    const picker = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      quality: 0.8,
    });

    if (!picker.cancelled) {
      setImage(picker.uri);
      setResult(null);
      setMaskUrl(null);
    }
  }
```

*** End Patch